Building a Pipeline for Time-Series Data

Hourly traffic forecasting on Interstate 94 (Minneapolis–St Paul)

Machine Learning Pipeline · Formative 1 · Group 21

GitHub repository: https://github.com/jagonzibwa/Formative-1-Group-21

Date: 26 October 2026

Team

Member	Role
Member 1	Task 1A/B - EDA & preprocessing
Member 2	Task 1C - Modelling
Member 3	Task 2 - Databases
Member 4	Task 3 & 4 - API and prediction

Summary

- **Problem:** forecast next-hour westbound traffic volume on I-94 from traffic history, calendar and weather.
- **Data:** Kaggle *Metro Interstate Traffic Volume*, 48,204 hourly rows, Oct 2012 – Sep 2018, with weather and holiday variables.
- **Model:** Random Forest (tuned) with lag and moving-average features. Test MAE **140 vehicles/hour** (R^2 0.988), a 59% lower error than the seasonal-naive baseline (340).
- **Storage:** a 4-table MySQL schema and a validated MongoDB collection, each with 5–6 analytical queries.
- **Serving:** a FastAPI service with matching CRUD, latest-record and date-range endpoints for both databases, plus a prediction script that pulls data from the API (23 automated API checks, 23 passing).

1. Problem definition and dataset

Problem. Interstate 94 between Minneapolis and St Paul is one of Minnesota's busiest corridors. Traffic managers need short-term forecasts of volume to time ramp meters, position incident-response crews, schedule roadwork outside busy hours and warn drivers of congestion. We frame this as a **one-step-ahead regression forecast**: given everything known up to hour *t−1* (past traffic, the calendar, and the weather), predict the number of westbound vehicles in hour *t*.

Dataset. *Metro Interstate Traffic Volume* (Kaggle, mirrored from the UCI Machine Learning Repository #492, CC BY 4.0). Traffic counts come from MnDOT automatic traffic recorder 301; hourly weather comes from OpenWeatherMap.

Requirement	How the dataset meets it
Clear timestamp	`date_time`: hourly, 2012-10-02 09:00 → 2018-09-30 23:00
Meaningful target	`traffic_volume`: vehicles per hour, 0–7,280; directly used in traffic operations
Multiple variables over time	temperature (K), rain and snow (mm/h), cloud cover (%), weather category and description, holiday name

Why this dataset. It has a target that matters in practice and plausible external drivers (weather, holidays) to test. It is also *messy in realistic ways*: duplicate timestamps, 22.8% missing hours including a 10-month outage, and physically impossible sensor values. That makes the preprocessing stage a real engineering problem instead of a formality, and it fits a relational and a document database well (one hour can carry several weather conditions).

2. Task 1A — Understanding the dataset

All exploration is in `notebooks/01_eda_preprocessing.ipynb`. Cleaning and feature code lives in one module, `src/preprocessing.py`, which the training notebook and the prediction script also import. Every stage therefore transforms data in exactly the same way.

2.1 Time range, frequency and duplicates

- **Time range:** 2 Oct 2012 09:00 → 30 Sep 2018 23:00 (2,189 days, ≈ 6 years).
- **Granularity:** hourly. 37,986 of the steps between consecutive unique timestamps are exactly 1 hour; the other step sizes are gaps.
- **Duplicates:** 7,629 rows share a timestamp (up to 6 rows per hour). Inside each group `traffic_volume` is identical; only the weather condition differs. The source adds a row per extra condition, for example *light rain* + *drizzle*. We collapse to one row per hour and keep the most severe condition (Thunderstorm > Squall > Snow > Rain > … > Clear).

2.2 Missing values and how they were handled

The only explicit nulls are in `holiday`, which means “no holiday”. The holiday name is also written only on the 00:00 row. The larger problem is **implicit**: 11,976 of 52,551 expected hourly slots (22.8%) have no row. There are 2,588 gaps: 2,192 are a single hour, 260 are 2–3 hours, and one outage runs from 8 Aug 2014 to 11 Jun 2015 (7,386 hours).

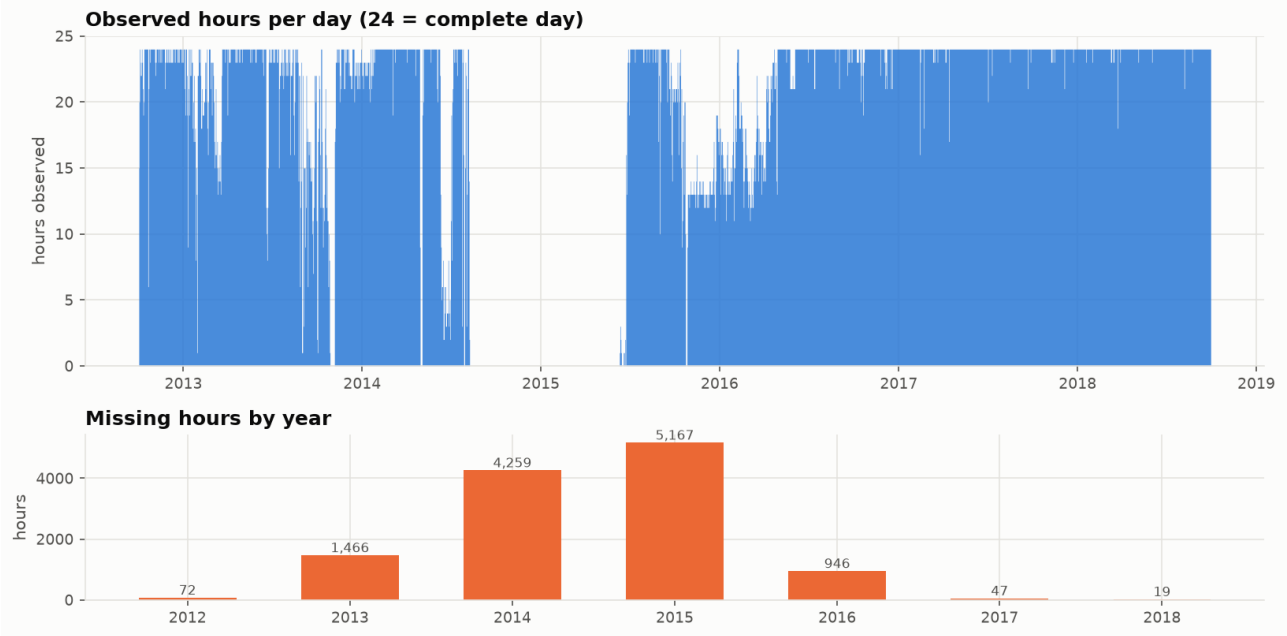


Figure 1 – Observed hours per day and missing hours per year. The 2014–2015 outage is the dominant gap.

Problem	Method	Why this method
Holiday only on 00:00 row	Spread the name to all 24 hours of that date; add binary is_holiday	A holiday is a property of the day; leaving 23 hours unlabelled hides its effect
Gaps ≤ 3 h (2,452 gaps)	Time-based linear interpolation (numeric), forward-fill (weather category); flag is_imputed = 1	Traffic and weather change smoothly over 1–3 h; a regular grid keeps shift()-based lags correct; the flag keeps it auditable
Gaps > 3 h, incl. the 10-month outage	Not imputed. Rows whose lag/rolling windows touch them are dropped	Filling days or months would invent daily and weekly cycles and leak fake values into lag features
Global mean / median fill	Rejected	Traffic swings 300 → 6,000 within hours; a constant destroys the daily cycle

Invalid values. Ten rows report 0 K (−273 °C); the coldest plausible reading is −29.8 °C. One hour reports 9,831 mm of rain, while the next-highest value is 55.6 mm. Both are set to NaN and handled with the rules above. The temperature failures come in 4–6 hour runs, so those rows are dropped rather than interpolated. Result: 48,204 raw rows → 40,575 unique hours → a 52,551-hour regular grid (2,771 imputed hours) → **36,568 model-ready rows** with 33 features.

2.3 Statistical distribution of numerical columns

Column	Mean	Std	Min	Median	Max	Skew	% zero
traffic_volume	3,290.7	1,984.8	0	3,427	7,280	−0.11	0.0
temp (°C)	8.24	13.09	−29.76	9.72	36.92	−0.38	0.1
rain_1h (mm)	0.08	0.77	0	0	55.63	28.64	94.9
snow_1h (mm)	0.00	0.01	0	0	0.51	65.91	99.9
clouds_all (%)	44.2	38.7	0	40	100	0.03	4.8

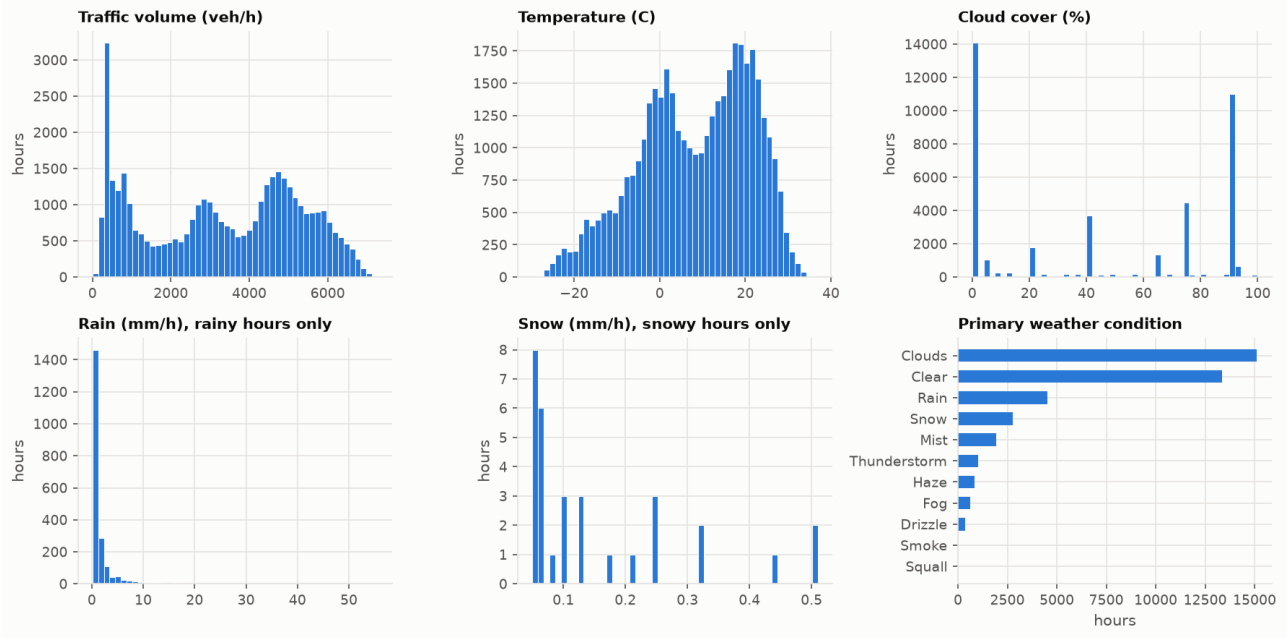


Figure 2 – Distributions of the numerical columns and of the primary weather condition.

Interpretation. The target is **bimodal**: a night-time mode near 300–1,000 vehicles/h and a daytime plateau near 4,500–6,000. That is a first sign that hour of day explains most of the variance. Temperature is wide and roughly symmetric (Minnesota seasons). Rain and snow amounts are zero-inflated and extremely skewed: only 5% of hours have rain and 0.08% have measured snow, so the categorical `weather_main = Snow` (≈ 7% of hours) carries more information than `snow_1h`. Cloud cover is U-shaped. Tree models do not need normal inputs, so no transformation was applied.

3. Task 1B — Analytical questions

Six questions were analysed, each with at least one visualisation. **Q1, Q3 and Q6 use moving averages and/or lagged features**, which the brief requires for at least two questions.

Q1. Does traffic have a long-term trend? (moving averages)

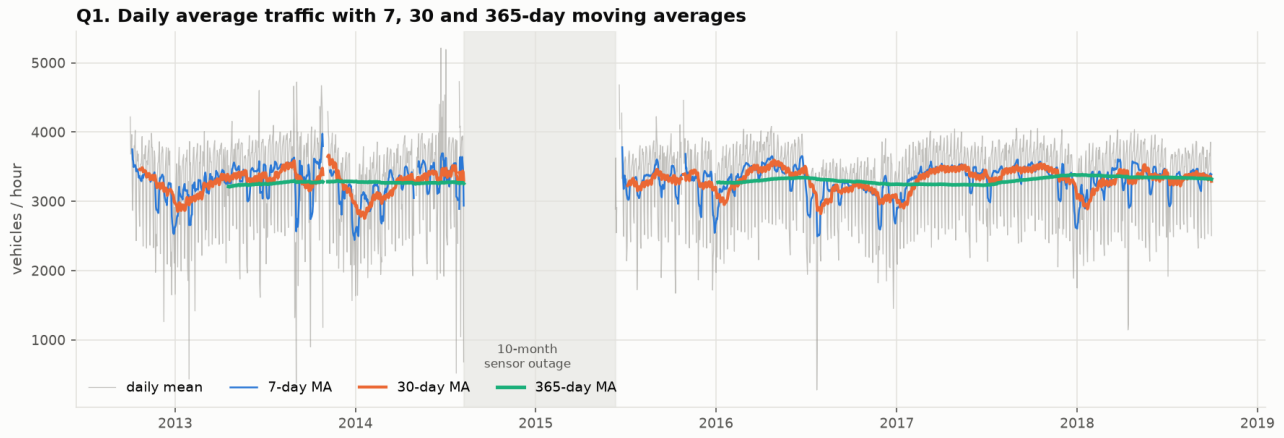


Figure 3 – Daily mean traffic with 7-, 30- and 365-day moving averages.

Interpretation. The 7-day MA still shows weekly and holiday dips. The 30-day MA shows a winter dip each December–January (≈ 3,050 vehicles/h). The 365-day MA is almost flat at about 3,200–3,380. A linear fit on daily means gives **+14.6 vehicles/h per year (+0.44%/year, p = 0.055)**, not significant at 5%. Conclusion: **no meaningful trend**; the series is level-stationary with strong seasonality, so no de-trending is needed. A model trained on 2012–2017 should carry over to 2018.

Q2. What daily and weekly seasonal patterns exist?

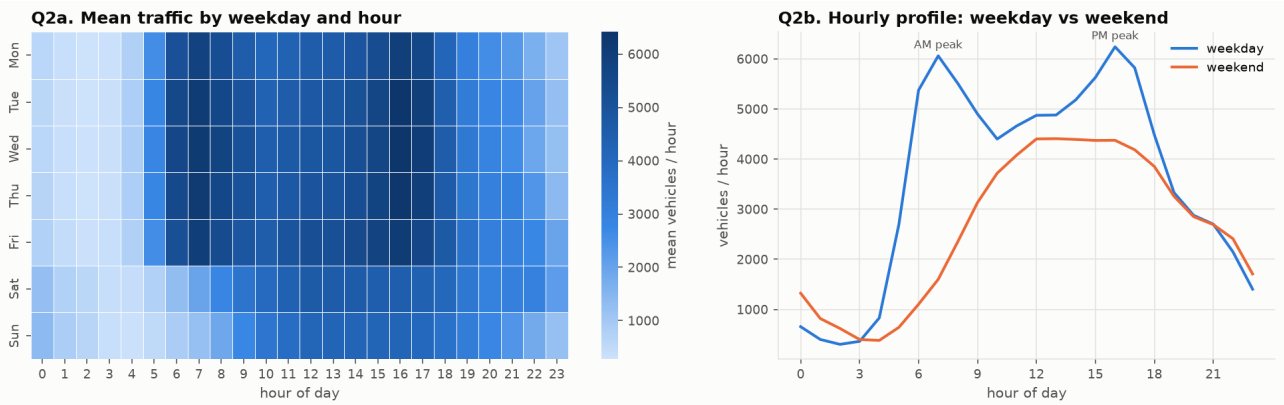


Figure 4 – Mean traffic by weekday × hour, and weekday vs weekend hourly profiles.

Interpretation. Traffic bottoms out at 03:00 (~350/h). Weekdays have a commuter double peak: **07:00 (~6,060)** and **16:00 (~6,240)**, with a ~4,800 midday plateau. Weekends have no morning rush, only a broad 12:00–15:00 peak; the weekend mean is **26% lower** (2,624 vs 3,557). The annual cycle is mild (Dec/Jan lowest). Hour × weekday is the main structure, so the model gets hour, weekday, weekend flags and cyclical encodings.

Q3. Are there lag effects? (lagged features)

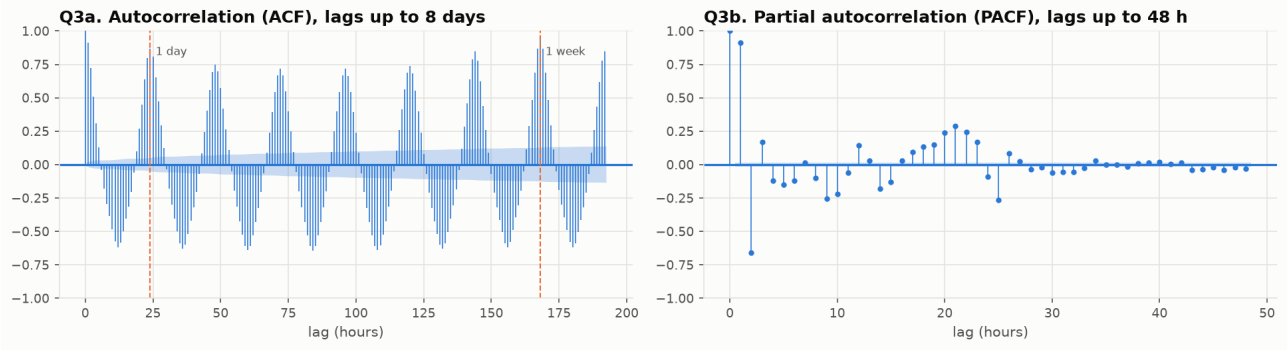


Figure 5 – ACF up to 8 days and PACF up to 48 hours.

Interpretation. Autocorrelation is **0.915 at lag 1 h**, falls to 0.52 at 3 h and to **−0.62 at 12 h** (peak vs trough). It returns to 0.863 at 24 h and peaks at **0.946 at 168 h**: the same hour last week is the single best predictor, because it matches both hour and weekday. The PACF has a large positive spike at lag 1, a large negative spike at lag 2 and a cluster at 20–25 h. As a forecast, *same as last week* scores MAE ≈ 349 and *same as an hour ago* ≈ 579. We therefore use lags 1, 2, 3, 24 and 168 h, and lag 168 h is the baseline to beat.

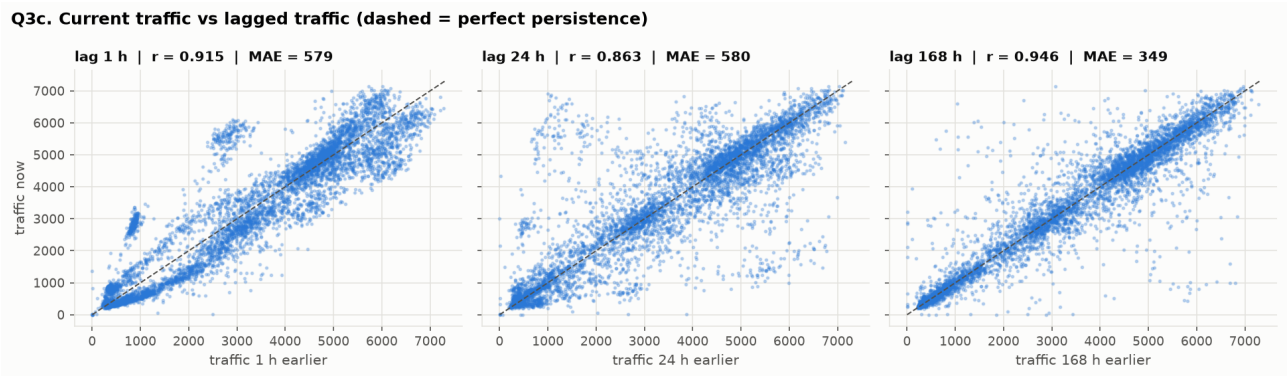


Figure 5b – Current traffic vs traffic 1 h, 24 h and 168 h earlier (Q3, continued).

Q4. Do weather variables correlate with traffic?

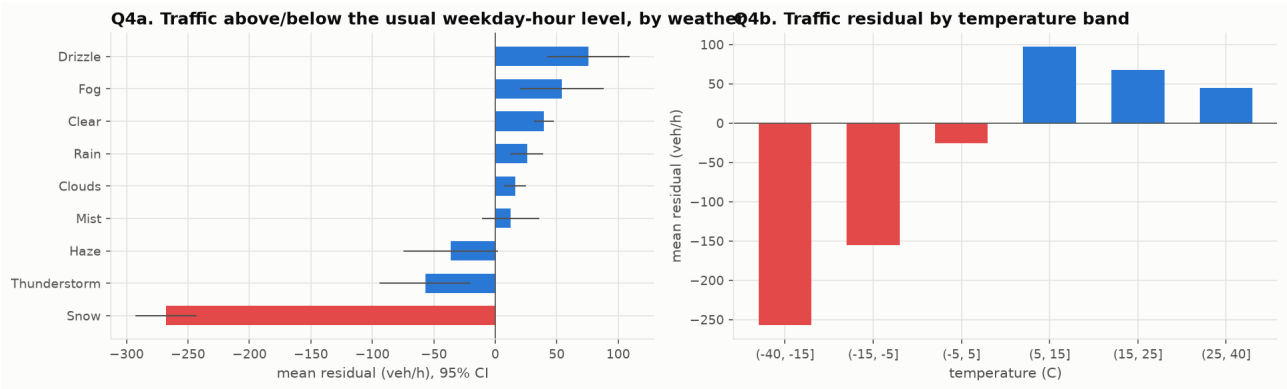


Figure 6 – Traffic residual (actual minus the usual weekday-hour level) by weather condition and temperature band.

Interpretation. Raw correlations are tiny (temperature r = 0.13, rain −0.01) because hour of day masks them. After subtracting the usual weekday-hour level, **snow is the only condition with a large effect (≈ −270 vehicles/h)**. In the weekday rush hour, snowy hours carry **12% fewer vehicles** (5,210 vs 5,925 when clear). Very cold hours (< −15 °C) are ≈ 260 below normal. Rain, mist and fog effects are within noise. Weather is a **secondary driver**, kept mainly to catch snow days.

Q5. How do holidays change traffic?

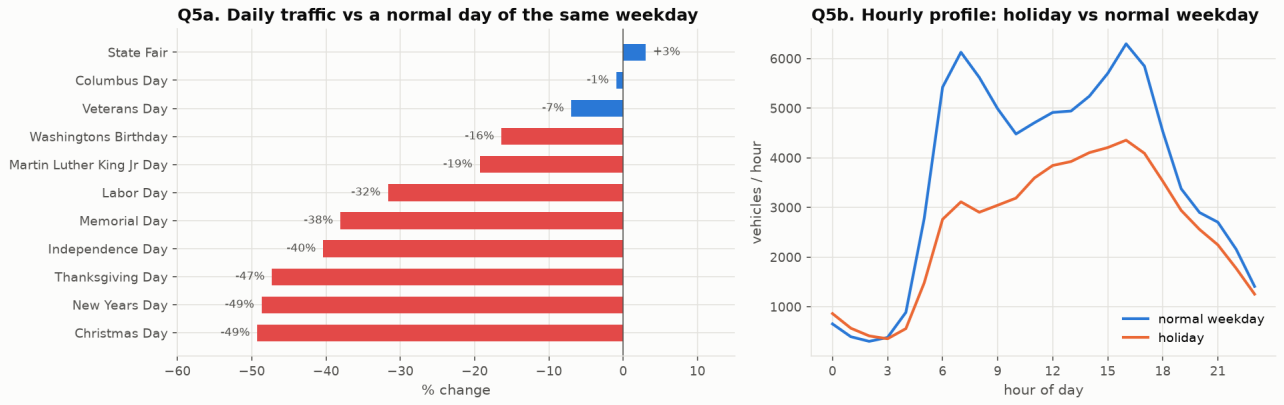


Figure 7 – Daily traffic on each holiday vs a normal day of the same weekday, and hourly profiles.

Interpretation. Holidays cut daily traffic by **27% on average** (53 complete holiday days). Christmas, New Year and Thanksgiving are ≈ −47% to −49%; Independence, Memorial and Labor Day −32% to −40%. MLK and Presidents' Day are −16% to −19%, Veterans Day −7%. Columbus Day (≈ 0%) and the Minnesota State Fair (+3%) show no drop. On holidays the morning peak disappears and the profile looks like a weekend. The effect varies by holiday and each one appears only ~5 times, so holidays stay a source of error.

Q6. Do recent moving averages tell us whether this hour will be above or below last week? (lags + moving averages)

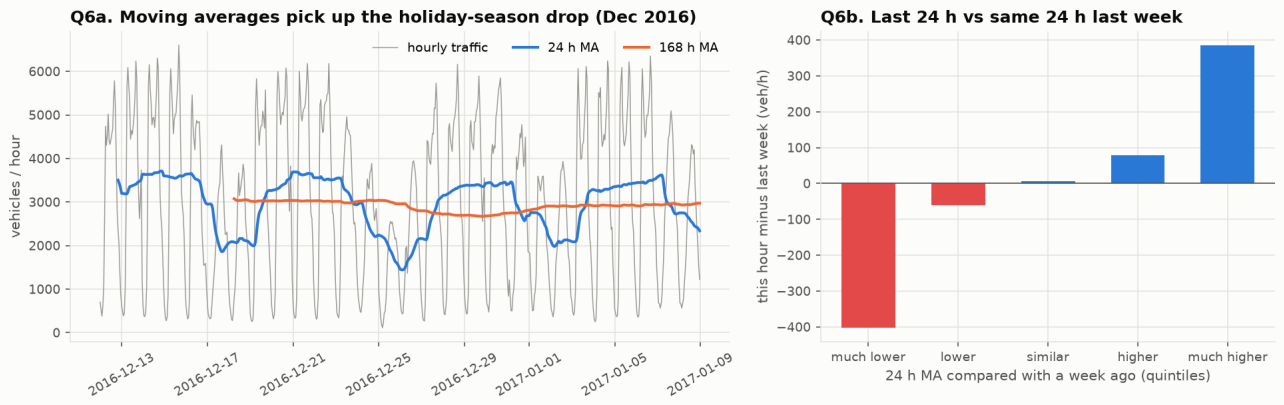


Figure 8 – 24 h and 168 h moving averages around Christmas 2016, and deviation from last week by quintile of the 24 h MA deviation.

Interpretation. Alone, long moving averages correlate weakly with the hourly target (24 h MA r ≈ 0.19); they remove the daily cycle on purpose. Combined with the weekly lag they work: when the last 24 h ran well below the same 24 h last week (bottom quintile), the current hour is on average **≈ 400 vehicles/h below** last week's value, and ≈ 390 above in the top quintile (r = 0.47). Scaling last week's value by the ratio of the two 24 h MAs cuts the seasonal-naive MAE from **350 to 327 (−6.4%)** with no model at all. So we include 3 h, 24 h and 168 h moving averages and a 24 h rolling standard deviation.

4. Task 1C — Model training and experiments

Features (33). Calendar: hour, weekday, month, weekend, holiday, sine/cosine encodings of hour and weekday. Weather: temperature (°C), rain, snow, cloud cover, 11 weather one-hots. Lags: 1, 2, 3, 24 and 168 h. Moving averages: 3, 24 and 168 h means and a 24 h standard deviation. Every lag and rolling feature is shifted by at least one hour, so no feature can see the value being predicted.

Validation design. A chronological hold-out: training 2012-10 → 2017-12 (30,190 rows), test 2018-01 → 2018-09 (6,378 rows). Hyperparameters were tuned with **TimeSeriesSplit** (4 expanding folds) inside the training period, so every validation fold lies in the future of its training data. The final model is chosen by **cross-validated MAE**; the test set is only used for reporting. Metrics: MAE, RMSE, R^2 and WAPE. MAPE was avoided because night-time volumes near zero make it explode.

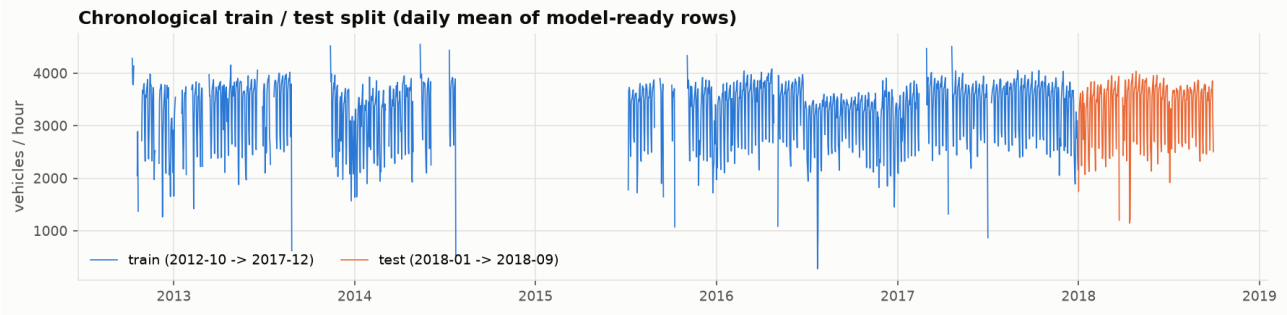


Figure 9 – Chronological train / test split.

Experiment table

Exp	Model	Features	Tuning	CV MAE	Test MAE	Test RMSE	Test R^2	WAPE %
E1	Seasonal naive (lag 168 h)	lag_168 only	none	352.9	340.2	651.2	0.891	10.25
E2	Linear Regression	all 33	none	295.2	281.8	397.7	0.959	8.49
E3	Ridge (tuned)	all 33	GridSearchCV, 6 values x 4 folds	294.7	281.7	398.0	0.959	8.49
E4	Random Forest (default)	all 33	none	184.4	146.4	231.4	0.986	4.41
E5	Random Forest (tuned)	all 33	RandomizedSearchCV, 8 configs x 4 folds	173.8	140.0	219.4	0.988	4.22
E6	HistGradientBoosting (default)	all 33	none	181.7	149.7	229.3	0.987	4.51
E7	HistGradientBoosting (tuned)	all 33	RandomizedSearchCV, 25 configs x 4 folds	174.5	139.7	218.1	0.988	4.21
E8	HGB tuned - no lags/MAs	24 (drops 9)	reuses E7	301.4	222.1	377.4	0.963	6.69
E9	HGB tuned - no weather	18 (drops 15)	reuses E7	174.5	140.0	219.8	0.988	4.22

Highlighted: the model selected by lowest CV MAE and saved to models/traffic_model.joblib.

Exp	Best hyperparameters found
E3	{ridge__alpha: 10}
E5	{n_estimators: 150, min_samples_leaf: 2, max_samples: None, max_features: 0.33, max_depth: 18}
E7	{l2_regularization: 0.0053, learning_rate: 0.0305, max_features: 0.6521, max_iter: 221, max_leaf_nodes: 122, min_samples_leaf: 58}

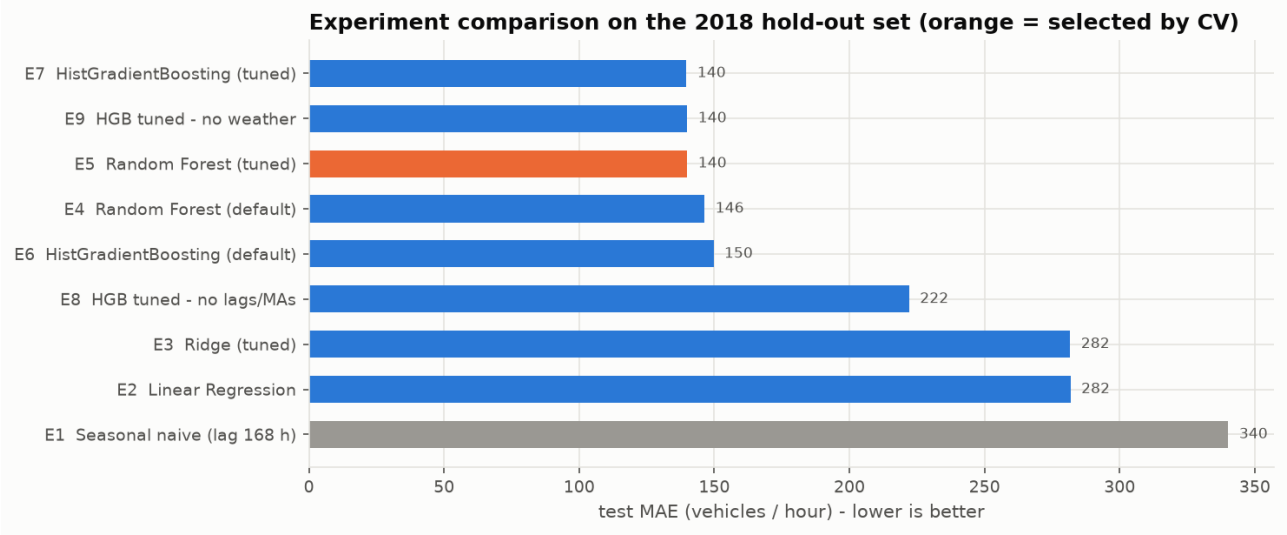


Figure 10 – Test MAE per experiment (orange = selected model, grey = baseline).

Findings

- **Every trained model beats the seasonal-naive baseline.** The tuned tree models cut MAE from 340 to ≈ 140 vehicles/h (−59%), a WAPE of 4.2%. RMSE drops 651 → 219, so large misses are much rarer.
- **Linear models plateau at MAE ≈ 282.** The hour × weekday × lag interactions are non-linear; the Ridge grid is flat between α = 0.01 and 100.
- **Tuning improves both tree families:** Random Forest CV MAE 184.4 → 173.8 (−5.7%), with depth-limited trees (max_depth 18) and 33% of features per split. Gradient boosting 181.7 → 174.5 (−4.0%), with a lower learning rate (≈ 0.03) and larger leaves. The search shows learning rates above ≈ 0.18 are clearly worse.
- **E5 and E7 are practically tied** (0.4% apart). E5 is kept because the pre-set rule is lowest CV MAE; E7 is an equally valid, ≈ 30× smaller alternative.
- **Ablations confirm the EDA.** Without lags and moving averages (E8), CV MAE rises 73% (174.5 → 301.4). Without weather (E9) it is unchanged: snow is rare and its effect shows up in lag_1 within an hour.

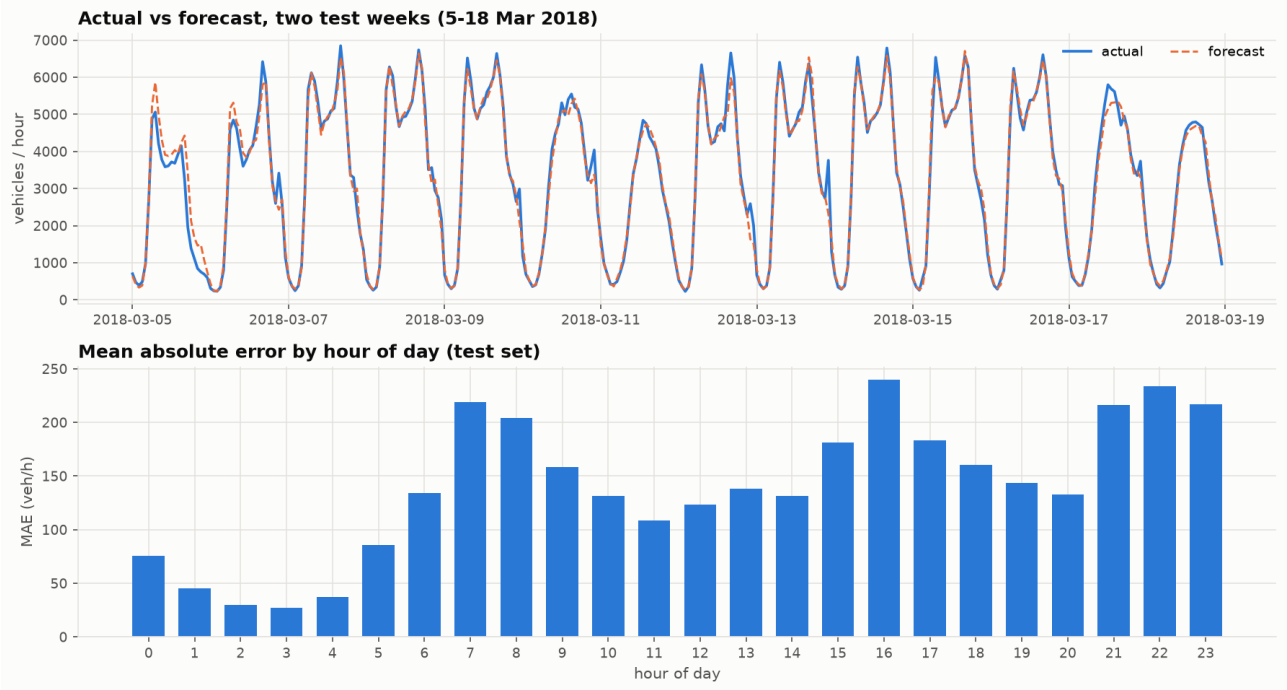


Figure 11 – Actual vs forecast for two test weeks, and error by hour of day.

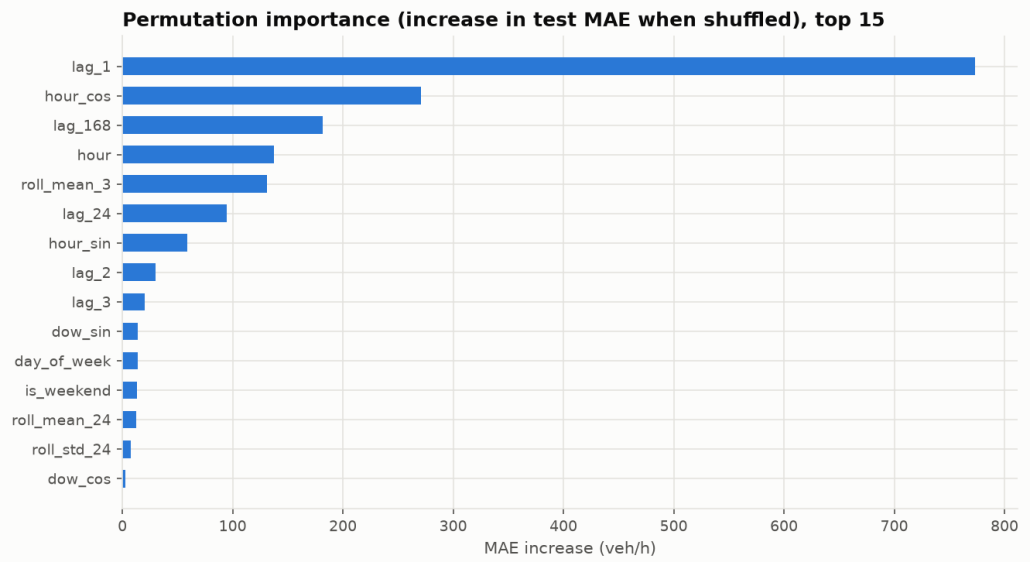


Figure 12 – Permutation importance on the test set (top 15).

Where the model fails. Errors peak at the 07:00–08:00 and 16:00 rush-hour ramps and at 21:00–23:00. Four of the five worst test days are heavy-snow days (19–23 snowy hours each, e.g. the 14 Apr 2018 storm, with volume 64% below the same day last week). Lagged traffic only adapts after the drop starts. Next steps would be a snow-intensity feature from a weather forecast, an events calendar, multi-step horizons and an LSTM for comparison.

5. Task 2 — Database design

Both databases store the **raw** 40,575 hourly observations exactly as published, including the invalid readings. Cleaning happens in the Python pipeline, so the databases remain the unmodified source of truth and the prediction script re-applies the same preprocessing to whatever the API returns. Loaders: `database/load_mysql.py` and `load_mongodb.py`.

5.1 Relational schema (MySQL 8.4)

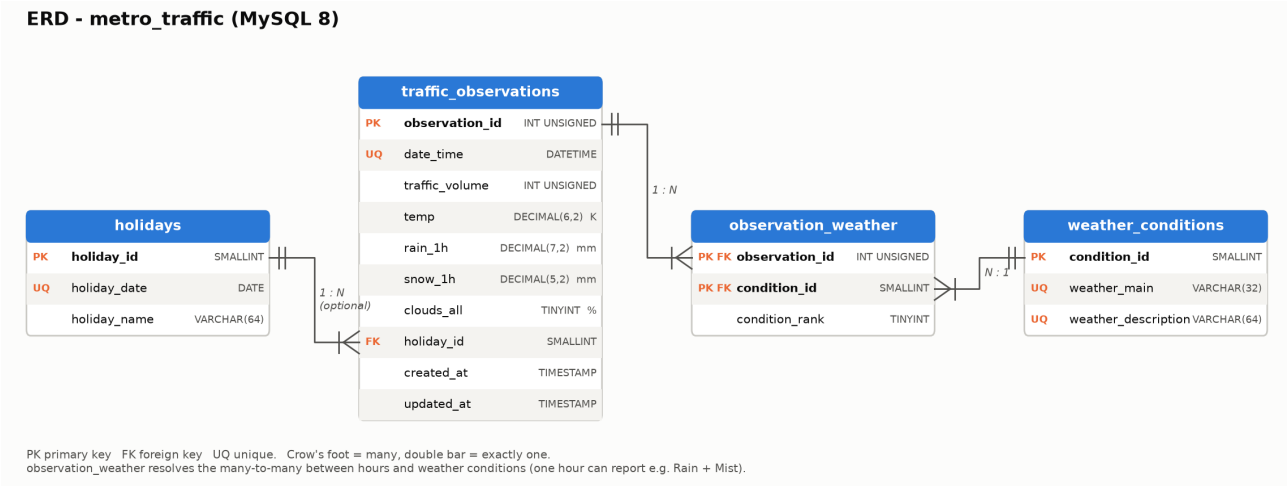


Figure 13 – Entity-relationship diagram (database/sql/erd.png).

Table	Rows	Role
traffic_observations	40,575	Time-series fact table, one row per hour. UNIQUE(date_time) enforces one row per hour and serves every range / latest query.
weather_conditions	38	Lookup of distinct (main, description) pairs; case-sensitive collation because the source has both “Sky is Clear” and “sky is clear”.
observation_weather	48,123	Junction table: an hour can report several conditions, so hour ↔ condition is many-to-many. condition_rank keeps source order (1 = primary).
holidays	53	One row per holiday date; all 24 hours of that date reference it.

Integrity rules: FKs with ON DELETE CASCADE (deleting an hour removes its weather links) and SET NULL for holidays; CHECK constraints on cloud cover and precipitation; created_at / updated_at audit columns; a denormalised view `v_hourly_traffic`. Excerpt of `database/sql/schema.sql`:

```
CREATE TABLE traffic_observations (
    observation_id   INT UNSIGNED      NOT NULL AUTO_INCREMENT,
    date_time        DATETIME          NOT NULL,
    traffic_volume   INT UNSIGNED      NOT NULL,
    temp             DECIMAL(6,2)      NULL,      -- Kelvin, stored raw
    rain_1h          DECIMAL(7,2)      NOT NULL DEFAULT 0,
    snow_1h          DECIMAL(5,2)      NOT NULL DEFAULT 0,
    clouds_all       TINYINT UNSIGNED  NOT NULL,
    holiday_id       SMALLINT UNSIGNED NULL,
    PRIMARY KEY (observation_id),
    UNIQUE KEY uq_observation_time (date_time),
    CONSTRAINT fk_observation_holiday FOREIGN KEY (holiday_id)
        REFERENCES holidays (holiday_id) ON DELETE SET NULL,
    CONSTRAINT chk_clouds CHECK (clouds_all BETWEEN 0 AND 100)
);
CREATE TABLE observation_weather (
    observation_id INT UNSIGNED NOT NULL, condition_id SMALLINT UNSIGNED NOT NULL,
    condition_rank TINYINT UNSIGNED NOT NULL DEFAULT 1,
    PRIMARY KEY (observation_id, condition_id),
    FOREIGN KEY (observation_id) REFERENCES traffic_observations (observation_id) ON DELETE CASCADE,
    FOREIGN KEY (condition_id)   REFERENCES weather_conditions (condition_id)
);
```

5.2 MongoDB collection design

Collection `metro_traffic.traffic_readings`, one document per hour:

```
{
  "_id":            {"$oid": "6adf49388b5edaeb3c093fca"},
  "timestamp":      {"$date": "2012-10-10T07:00:00Z"},
  "traffic_volume": 6793,
  "weather": {
    "temp_k": 281.25, "rain_1h_mm": 0.0, "snow_1h_mm": 0.0, "clouds_pct": 99,
    "conditions": [ {"main": "Rain",    "description": "light rain"},
                    {"main": "Drizzle", "description": "light intensity drizzle"} ]
  },
  "holiday":  null,
  "calendar": {"year": 2012, "month": 10, "day": 10, "hour": 7, "day_of_week": 2, "is_weekend": false}
}
```

- **Embed rather than reference:** weather conditions and the holiday are always read with the hour, so the SQL junction table becomes a small embedded array and every read is a single document.
- **Pre-computed calendar sub-document** makes hour-of-day and weekday aggregations simple `$groups` backed by an index. The API rebuilds it on every write so it cannot drift.
- **Regular collection with a unique index on timestamp** rather than a native time-series collection, which cannot have unique indexes. Here one document per hour must be guaranteed, with full CRUD by id.
- **Indexes:** unique timestamp, (day_of_week, hour), multikey on weather.conditions.main, partial index on holiday. A **$jsonSchema validator** enforces types, ranges and at least one condition.
- Four complete sample documents are in `database/mongodb/sample_documents.json`.

5.3 Queries and results

Six SQL and five MongoDB queries were run (`database/run_queries.py`). Full queries and outputs are in `outputs/query_results/`; selected results follow.

#	SQL query	Technique
Q1	Latest 5 observations with conditions and holiday	3-table JOIN, GROUP_CONCAT
Q2	Hour-of-day profile, weekday vs weekend	Conditional aggregation
Q3	Monthly summary for 2017	Date-range filter, GROUP BY month
Q4	Holiday effect vs an average non-holiday day	CTE + scalar subquery
Q5	Daily totals, 7-day MA, day-over-day and week-over-week change	Window functions AVG() OVER, LAG()
Q6	Weekday rush-hour volume by weather	Junction-table JOIN, HAVING

#	MongoDB query	Technique
M1	Latest reading	find().sort({timestamp: -1}).limit(1)
M2	Readings in a date range	$gte / $lte on timestamp
M3	Weekday rush-hour volume by primary weather	$match → $first → $group
M4	Daily totals + 7-day moving average	$dateTrunc, $setWindowFields, $shift
M5	Holiday summary	$group on an embedded field

SQL Q1 – Latest 5 observations

```
SELECT  o.date_time,
        o.traffic_volume,
        ROUND(o.temp - 273.15, 1)                                                    AS temp_c,
        o.clouds_all,
        GROUP_CONCAT(wc.weather_description ORDER BY ow.condition_rank SEPARATOR ', ') AS conditions,
        COALESCE(h.holiday_name, '-')                                                AS holiday
FROM    traffic_observations o
JOIN    observation_weather ow ON ow.observation_id = o.observation_id
JOIN    weather_conditions  wc ON wc.condition_id   = ow.condition_id
LEFT JOIN holidays          h  ON h.holiday_id      = o.holiday_id
GROUP BY o.observation_id, o.date_time, o.traffic_volume, o.temp, o.clouds_all, h.holiday_name
ORDER BY o.date_time DESC
LIMIT 5;
```

date_time	traffic_volume	temp_c	clouds_all	conditions	holiday
2018-09-30 23:00:00	954	9	90	overcast clouds	-
2018-09-30 22:00:00	1450	8.9	90	overcast clouds	-
2018-09-30 21:00:00	2159	9.6	90	proximity thunderstorm	-
2018-09-30 20:00:00	2781	9.6	90	overcast clouds	-
2018-09-30 19:00:00	3543	10.3	75	broken clouds	-

SQL Q5 – Daily totals with 7-day moving average (window functions)

```
WITH daily AS (
    SELECT DATE(date_time) AS day, SUM(traffic_volume) AS day_volume, COUNT(*) AS hours
    FROM   traffic_observations
    WHERE  date_time >= '2016-11-24' AND date_time < '2017-01-01'
    GROUP BY DATE(date_time)
)
SELECT  day,
        hours,
        day_volume,
        ROUND(AVG(day_volume) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)) AS ma_7day,
        day_volume - LAG(day_volume, 1) OVER (ORDER BY day)                                 AS change_vs_prev_day,
        day_volume - LAG(day_volume, 7) OVER (ORDER BY day)                                 AS change_vs_last_week
FROM    daily
ORDER BY day DESC
LIMIT 14;
```

day	hours	day_volume	ma_7day	change_vs_prev_day	change_vs_last_week
2016-12-31	24	64473	67419	-17261	10658
2016-12-30	24	81734	65896	-632	11124
2016-12-29	24	82366	64307	1344	-2554
2016-12-28	24	81022	64672	5404	-4380
2016-12-27	24	75618	65297	23776	-12916
2016-12-26	24	51842	67143	16967	-26471
2016-12-25	24	34875	70924	-18940	-5469
2016-12-24	24	53815	71705	-16795	4356

SQL Q4 – Holiday effect

```
WITH daily AS (
    SELECT  DATE(o.date_time)        AS day,
            SUM(o.traffic_volume)    AS day_volume,
            MAX(h.holiday_name)      AS holiday_name
    FROM    traffic_observations o
    LEFT JOIN holidays h ON h.holiday_id = o.holiday_id
    GROUP BY DATE(o.date_time)
    HAVING  COUNT(*) = 24
)
SELECT  holiday_name,
        COUNT(*)                     AS complete_days,
        ROUND(AVG(day_volume))       AS avg_daily_volume,
        ROUND(100 * (AVG(day_volume)
             / (SELECT AVG(day_volume) FROM daily WHERE holiday_name IS NULL) - 1), 1) AS pct_vs_normal_day
FROM    daily
WHERE   holiday_name IS NOT NULL
GROUP BY holiday_name
ORDER BY pct_vs_normal_day;
```

holiday_name	complete_days	avg_daily_volume	pct_vs_normal_day
New Years Day	3	43996	-44.7
Christmas Day	3	45600	-42.7
Thanksgiving Day	2	48329	-39.2
Memorial Day	4	50323	-36.7
Independence Day	5	51724	-35
Labor Day	4	55805	-29.8
Martin Luther King Jr Day	2	66414	-16.5
Washingtons Birthday	4	66891	-15.9
Veterans Day	3	80473	1.2
Columbus Day	2	83027	4.4
State Fair	4	91280	14.8

MongoDB M1 – Latest reading

```
db.traffic_readings.find({}, {_id: 0, timestamp: 1, traffic_volume: 1, "weather.conditions": 1, holiday: 1})
  .sort({timestamp: -1}).limit(1)
```

timestamp	traffic_volume	holiday	weather.conditions
2018-09-30 23:00:00	954		Clouds / overcast clouds

MongoDB M2 – Records in a date range

```
db.traffic_readings.find(
  {timestamp: {$gte: ISODate("2018-09-24T06:00:00Z"), $lte: ISODate("2018-09-24T10:00:00Z")}},
  {_id: 0, timestamp: 1, traffic_volume: 1, "weather.temp_k": 1, "weather.conditions.main": 1}
).sort({timestamp: 1})
```

timestamp	traffic_volume	weather.temp_k	weather.conditions
2018-09-24 06:00:00	5747	287.33	Thunderstorm
2018-09-24 07:00:00	6591	287.18	Clouds
2018-09-24 08:00:00	5900	287.19	Clouds
2018-09-24 09:00:00	4936	288.25	Clear
2018-09-24 10:00:00	4351	290.03	Clouds

MongoDB M4 – Daily totals with 7-day moving average ($setWindowFields)

```
db.traffic_readings.aggregate([
  {"$match": {"timestamp": {"$gte": "ISODate('2016-11-24T00:00:00Z')", "$lt":
      "ISODate('2017-01-01T00:00:00Z')"}}},
  {"$group": {"_id": {"$dateTrunc": {"date": "$timestamp", "unit": "day"}}, "day_volume": {"$sum":
      "$traffic_volume"}, "hours": {"$sum": 1}}},
  {"$setWindowFields": {"sortBy": {"_id": 1}, "output": {"ma_7day": {"$avg": "$day_volume", "window":
      {"documents": [-6, 0]}}, "prev_day": {"$shift": {"output": "$day_volume", "by": -1}}}}},
  {"$project": {"_id": 0, "day": {"$dateToString": {"date": "$_id", "format": "%Y-%m-%d"}}, "hours": 1,
      "day_volume": 1, "ma_7day": {"$round": ["$ma_7day", 0]}, "change_vs_prev_day": {"$subtract":
      ["$day_volume", "$prev_day"]}}},
  {"$sort": {"day": -1}},
  {"$limit": 14},
])
```

day_volume	hours	day	ma_7day	change_vs_prev_day
64473	24	2016-12-31	67419	-17261
81734	24	2016-12-30	65896	-632
82366	24	2016-12-29	64307	1344
81022	24	2016-12-28	64672	5404
75618	24	2016-12-27	65297	23776
51842	24	2016-12-26	67143	16967
34875	24	2016-12-25	70924	-18940
53815	24	2016-12-24	71705	-16795

MongoDB M3 – Weekday rush-hour volume by weather (same answer as SQL Q6)

```
db.traffic_readings.aggregate([
  {"$match": {"calendar.is_weekend": false, "calendar.hour": {"$in": [7, 8, 16, 17]}, "holiday": null}},
  {"$set": {"primary": {"$first": "$weather.conditions"}}},
  {"$group": {"_id": "$primary.main", "rush_hours": {"$sum": 1}, "avg_volume": {"$avg":
      "$traffic_volume"}}},
  {"$match": {"rush_hours": {"$gte": 30}}},
  {"$project": {"_id": 0, "weather_main": "$_id", "rush_hours": 1, "avg_volume": {"$round": ["$avg_volume",
      0]}}},
  {"$sort": {"avg_volume": -1}},
])
```

rush_hours	weather_main	avg_volume
33	Fog	6132

rush_hours	weather_main	avg_volume
58	Drizzle	6107
44	Thunderstorm	6068
1946	Clouds	6059
1200	Clear	6043
67	Haze	6037
416	Mist	6024
580	Rain	5990
246	Snow	5421

6. Task 3 — CRUD and time-series API

A **FastAPI** service (`api/`) exposes the same endpoints and the same JSON record shape for both databases, so a client can switch between `/sql` and `/mongo` without code changes. Interactive documentation is generated at `/docs`.

Method	Path (both /sql and /mongo)	Behaviour
POST	/{db}/records	Create an hourly record. 201; 409 if the hour already exists; 422 on invalid body
GET	/{db}/records?limit=&offset;=	Paginated list, newest first
GET	/{db}/records/latest	**Latest record** (most recent hour)
GET	/{db}/records/range?start=&end;=	**Records by date range**, oldest first; 400 if start > end
GET	/{db}/records/{id}	One record (SQL observation_id or Mongo ObjectId); 404 if missing
PUT	/{db}/records/{id}	Update only the fields sent; weather list replaced if provided
DELETE	/{db}/records/{id}	Delete and return the deleted record (SQL cascades weather links)
GET	/	Health check with record counts for both databases

Implementation notes

- **Validation:** Pydantic models enforce types and ranges (volume ≥ 0, cloud cover 0–100, temperature in Kelvin, at least one weather condition). MongoDB enforces its own $jsonSchema as a second line of defence.
- **SQL writes are transactional:** one POST/PUT touches up to four tables (get-or-create the holiday and weather lookups, insert the observation, link conditions) inside a single transaction.
- **Reads avoid the view:** range queries filter `traffic_observations` on its unique date_time index first, then fetch the weather for just those ids, rather than scanning a GROUP BY view.
- **MongoDB updates** rebuild the document so the derived calendar fields stay consistent; duplicate hours return 409 and validator failures return 422. Database outages return 503.

Testing

`scripts/test_api.py` runs a full create → read → latest → range → update → list → delete → read-404 cycle on each database, plus error cases. **23 of 23 checks pass**. Every request and response is logged in `outputs/api_test_results.md`.

Result	Request	Status (expected)
PASS	GET /	200 (200)
PASS	POST /sql/records	201 (201)
PASS	POST /sql/records	409 (409)

Result	Request	Status (expected)
PASS	GET /sql/records/40578	200 (200)
PASS	GET /sql/records/latest	200 (200)
PASS	GET /sql/records/range?start=2018-09-30T20:00:00&end;=2018-10-01T00:00:00	200 (200)
PASS	GET /sql/records/range?start=2018-10-02T00:00:00&end;=2018-10-01T00:00:00	400 (400)
PASS	PUT /sql/records/40578	200 (200)
PASS	PUT /sql/records/40578	422 (422)
PASS	GET /sql/records?limit=2	200 (200)
PASS	DELETE /sql/records/40578	200 (200)
PASS	GET /sql/records/40578	404 (404)
PASS	POST /mongo/records	201 (201)
PASS	POST /mongo/records	409 (409)
PASS	GET /mongo/records/6adf4b20c09ec4df5610bb7b	200 (200)
PASS	GET /mongo/records/latest	200 (200)
PASS	GET /mongo/records/range?start=2018-09-30T20:00:00&end;=2018-10-01T00:00:00	200 (200)
PASS	GET /mongo/records/range?start=2018-10-02T00:00:00&end;=2018-10-01T00:00:00	400 (400)
PASS	PUT /mongo/records/6adf4b20c09ec4df5610bb7b	200 (200)
PASS	PUT /mongo/records/6adf4b20c09ec4df5610bb7b	422 (422)
PASS	GET /mongo/records?limit=2	200 (200)
PASS	DELETE /mongo/records/6adf4b20c09ec4df5610bb7b	200 (200)
PASS	GET /mongo/records/6adf4b20c09ec4df5610bb7b	404 (404)

7. Task 4 — Prediction script

`scripts/predict.py` ties the previous tasks together in four steps:

Step	What happens
1. Fetch	GET /{db}/records/latest, then GET /{db}/records/range for the two weeks before it (the features need ≥ 169 hours of history).
2. Preprocess	API JSON is converted back to the raw-CSV layout and passed through the same src/preprocessing.py functions as training: invalid values → NaN, de-duplication, regular hourly grid, ≤ 3 h interpolation, calendar / weather / lag / moving-average features.
3. Load model	models/traffic_model.joblib plus its metadata; the script refuses to run if the saved feature list differs from the current pipeline.
4. Predict	Builds the feature row for the hour after the latest record (weather carried forward as a persistence forecast, holidays from the US federal calendar) and prints and saves the forecast to outputs/latest_forecast.json.

A `--at` option back-tests a known hour by hiding its value, and `--source sql|mongo` chooses the database. Both sources return the same forecast. As a consistency check, features built from API data for three test hours matched the training features to within 1e-11, and the predictions were identical.

```
$ python scripts/predict.py
[1/4] Fetching data from http://127.0.0.1:8000/mongo/records ...
      latest record: 2018-09-30T23:00:00  volume=954
      fetched 336 hourly records (2018-09-17T00:00:00 -> 2018-09-30T23:00:00)
[2/4] Preprocessing (clean -> hourly grid -> lag / moving-average features) ...
      grid: 336 hours, 0 imputed, 0 still missing
[3/4] Loading model ...
      Random Forest (tuned) (experiment E5, test MAE 140.0 veh/h)
[4/4] Forecasting ...
============================================================
 Forecast for Monday 01 Oct 2018, 00:00
   predicted traffic volume :    542 vehicles/hour
   previous hour            :    954
```

```
   same hour last week      :    509
============================================================

$ python scripts/predict.py --source sql --at 2018-09-28T17:00
 Forecast for Friday 28 Sep 2018, 17:00
   predicted traffic volume :  5,570 vehicles/hour
   actual traffic volume    :  5,695 vehicles/hour  (error 125)
```

8. Team contributions

Each member owned one part of the pipeline and committed their own work to the repository.

Member	Role	Major components
Member 1	Task 1A/B - EDA & preprocessing	Dataset choice and justification, missing-data strategy, invalid-value handling, six analytical questions and their visualisations (notebooks/01, src/preprocessing.py).
Member 2	Task 1C - Modelling	Feature engineering (lags, moving averages), time-series cross-validation, nine experiments, hyperparameter tuning, experiment table and model export (notebooks/02, models/).
Member 3	Task 2 - Databases	MySQL schema and ERD, MongoDB collection design and validator, loaders, SQL and MongoDB queries with results (database/).
Member 4	Task 3 & 4 - API and prediction	FastAPI CRUD and time-series endpoints for both databases, API test suite, prediction script that fetches from the API and forecasts (api/, scripts/).

9. Conclusion

The pipeline turns a messy six-year hourly dataset into a working forecasting service. EDA showed that traffic is driven by daily and weekly cycles and short-term momentum, while weather and holidays act as occasional shocks. That finding shaped the features (lags 1–168 h and moving averages) and is confirmed by the ablation experiments. The tuned tree models reach a 4.2% weighted error, 59% better than the seasonal-naive baseline. The same data is stored in a normalised MySQL schema and a document-oriented MongoDB collection, served through identical CRUD and time-series endpoints, and consumed by a prediction script that reuses the training pipeline unchanged. **Limitations:** the forecast horizon is one hour, next-hour weather is assumed to persist, and rare events (snowstorms, unusual holidays) remain the largest source of error.

Code, notebooks and instructions to reproduce every result:
https://github.com/jagonzibwa/Formative-1-Group-21